

EKSAMENSOPPGAVE

Bokmål

Emnekode: DAT108

Emnenavn: Programmering og webapplikasjoner

Klasse: 2. klasse DATA / INF

Dato: 9. desember 2024

Eksamensform: Skriftlig skoleeksamen (Wiseflow | FLOWmulti)

Eksamenstid: 4 timer (0900-1300)

Antall eksamensoppgaver: 7

Antall sider (medregnet denne): 15

Antall vedlegg: Ingen

Tillatte hjelpemiddel: Ingen

Lærere: Lars-Petter Helland (928 28 046), Bergen
 Bjarte Kileng (909 97 348), Bergen
 Erlend Raa Vågset (910 06 061), Førde
 Tord Waage Frøland (915 45 537), Haugesund

LYKKE TIL!

Oppgave 1 (10% ~ 24 minutter) – Strømmer

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutter) – Funksjonelle kontrakter

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 3 (10% ~ 24 minutter) – JavaScript teori

Flervalgsoppgave: Oppgavetekst i Wiseflow



Felles info for oppgave 4, 5, 6 og 7

Du skal svare på disse i Wiseflow. Ved å bruke kodevedlegg vil dere få uthevet syntaks og korrekte innrykk. Vi anbefaler ett vedlegg per oppgave. For å sikre automatisk lagring, følg denne oppskriften.

- 1 - Start med å gi vedlegget et navn. For eksempel «Oppgave 4.java».
- 2 - Klikk lagre. Da aktiverer du automatisk lagring.
- 3 - Start å skrive kode

Pass på at du har svart på alle oppgaver før du prøver å få oppgaver perfekte. Hvis du mener noe er uklart i oppgaven, presiser forutsetningene dine i besvarelsen.

Oppgavene begynner på neste side.

Oppgave 4 (15% ~ 36 minutter) – Tråder og trådsikkerhet

Nedenfor er det gitt et lite Java-program som simulerer et vintermarked der to Gløggkokere og to Pepperkakebakere jobber parallelt for å lage henholdsvis gløgg og pepperkaker.

Hver arbeider bruker 2 sekunder for å lage én enhet gløgg eller pepperkake, og legger deretter produktet på et felles serveringsbrett: `gløggServeringsbrett` eller `pepperkakeServeringsbrett`.

Disse serveringsbrettene deles mellom arbeiderne av samme type, slik at Gløggkokere og Pepperkakebakere legger sine varer på henholdsvis felles `gløggServeringsbrett` og `pepperkakeServeringsbrett`.

Tre kunder besøker markedet. Hver kunde velger tilfeldig mellom gløgg og pepperkaker, og tar fra brettet når varen er tilgjengelig. Hvis et brett er tomt, venter kunden før de prøver igjen. Alle arbeiderne og kundene kjører parallelt – kundene venter kun på at en vare skal bli tilgjengelig, ikke på hverandre.

Koden inneholder noen feil relatert til trådbehandling og trådsikring. Oppgavene vil veilede deg gjennom å identifisere og rette disse feilene.

Oppgaver

- a) **Kjøring av Arbeider- og Kundetråder.** I Vintermarked sin main-metode er arbeider- og kundetrådene definert.
 - i. Identifiser problemer med måten trådene er startet i main-metoden.
 - ii. Endre main-metoden slik at alle tråder kjører parallelt som forventet. (Svaret ditt trenger kun å ha med den delen av koden som er endret.)
- b) **Trådsikring for Serveringsbrettene.** I koden er `gløggServeringsbrett` og `pepperkakeServeringsbrett` implementert som Queue-objekter.
 - i. Forklar kort hva som kan skje hvis flere tråder får tilgang til serveringsbrettene samtidig uten trådsikring.
 - ii. Endre koden slik at bruken av serveringsbrettene blir trådsikker. (Svaret ditt trenger kun å ha med den delen av koden som er endret.)
- c) **Synkronisering av Tellere.** Koden inneholder følgende tellere:
 - Globale tellere for totalt antall produserte og konsumerte varer, som alle arbeiderne og kundene oppdaterer.
 - `gløggServert` for totalt antall gløgg servert.
 - `pepperkakerServert` for totalt antall pepperkaker servert.
 - Objektspesifikke tellere i hver Arbeider- og Kunde-instans som sporer hvor mange varer de individuelt har produsert eller konsumert.
 - `produkterLaget` i Arbeider for antall varer produsert av den enkelte arbeider.
 - `varerKonsumert` i Kunde for antall varer konsumert av den enkelte kunde.

Vurder hvilke av tellerne som trenger trådsikring og hvilke som ikke gjør det, og forklar kort hvorfor du mener det er nødvendig eller unødvendig.

- d) **Trådsikring av Tellere.** For de tellerne du identifiserte i forrige oppgave som krever trådsikring:
- Endre koden slik at disse tellerne håndterer trådsikker oppdatering. Velg en metode du mener er hensiktsmessig (for eksempel synchronized-blokker, Lock-objekter eller AtomicInteger).
 - Forklar kort hvorfor du valgte denne metoden.

e) **Forklare Dødlås eller Sult.**

Velg ett av følgende konsepter og forklar kort hva det innebærer:

- Dødlås (Deadlock): Beskriv hva dødlås er og gi et eksempel.
- Sult/utsulting (Starvation): Beskriv hva sult/utsulting er og gi et eksempel.

Start besvarelsen din ved å oppgi hvilket av de to konseptene du forklarer.

Kode:

```
public enum Produkt { GLOEGG, PEPPERKAKE; }
```

```
public class Vintermarked {

    // To felles serveringsbrett for hhv. gløgg og pepperkaker
    public static final Queue<String> gloeggServeringsbrett = new LinkedList<>();
    public static final Queue<String> pepperkakeServeringsbrett = new LinkedList<>();

    // Globale tellere for totalt antall servert
    public static int gloeggServert = 0;
    public static int pepperkakerServert = 0;

    public static void main(String[] args) {

        // Oppretter tråder for Gløggkokere og Pepperkakebakere
        Arbeider gloeggkoker1 = new Arbeider(
            "Gløggkoker-1", Produkt.GLOEGG, gloeggServeringsbrett);
        Arbeider gloeggkoker2 = new Arbeider(
            "Gløggkoker-2", Produkt.GLOEGG, gloeggServeringsbrett);
        Arbeider pepperkakebaker1 = new Arbeider(
            "Pepperkakebaker-1", Produkt.PEPPERKAKE, pepperkakeServeringsbrett);
        Arbeider pepperkakebaker2 = new Arbeider(
            "Pepperkakebaker-2", Produkt.PEPPERKAKE, pepperkakeServeringsbrett);

        // Starter arbeidernes prosesser
        gloeggkoker1.run();
        gloeggkoker2.run();
        pepperkakebaker1.run();
        pepperkakebaker2.run();

        // Oppretter og starter tråder for 3 kunder
        for (int i = 0; i < 3; i++) {
            new Thread(new Kunde(i)).start();
        }
    }
}
```

```

public class Arbeider implements Runnable {

    private final String navn;
    private final Produkt produkt;
    private final Queue<String> tilknyttetBrett;
    private int produkterLaget = 0;

    /* Konstruktør fjernet - Du kan anta at den finnes */

    @Override
    public void run() {
        try {
            while (true) {
                Thread.sleep(2000); // Produksjonstid på 2 sek

                tilknyttetBrett.add(produkt + " laget av " + navn);
                if (produkt == Produkt.GLOEGG) {
                    Vintermarked.gloeggServert++
                } else {
                    Vintermarked.pepperkakerServert++;
                }
                produkterLaget++;
                System.out.println(navn + " la til en "+ produkt +" på brettet. "
                    + "Totalt produsert av denne arbeideren: "
                    + produkterLaget);
            }
        } catch (InterruptedException e) { ... }
    }
}

```

```

public class Kunde implements Runnable {

    private final int id; // Unik ID for kunden
    private int varerKonsumert = 0;

    /* Konstruktør fjernet - Du kan anta at den finnes */

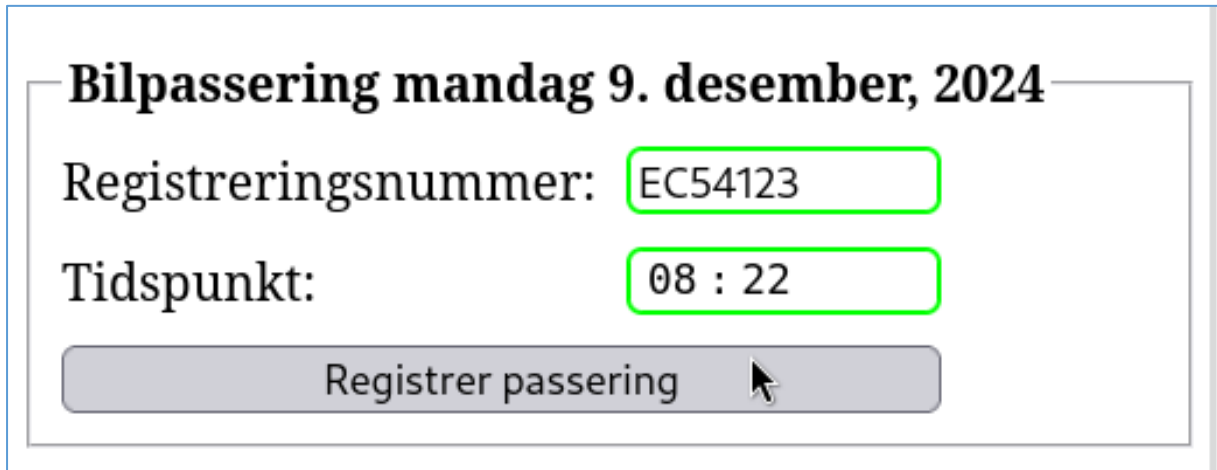
    @Override
    public void run() {
        try {
            while (true) {
                // Forsøker å ta gløgg fra brettet
                String vare = Vintermarked.gloeggServeringsbrett.poll();
                // Hvis tomt, forsøker å ta pepperkake
                if (vare == null) {
                    vare = Vintermarked.pepperkakeServeringsbrett.poll();
                }
                if (vare != null) {
                    varerKonsumert++;
                    System.out.println("Kunde " + id + " konsumerte: " + vare
                        + ". Totalt konsumert av denne kunden: "
                        + varerKonsumert);
                    Thread.sleep(1500); // Simulerer konsumtid på 1,5 sek
                } else {
                    System.out.println("Kunde " + id + " venter på vare...");
                    Thread.sleep(500); // Venter 0,5 sek hvis ingen varer er tilgjengelige
                }
            }
        } catch (InterruptedException e) { ... }
    }
}

```

Oppgave 5 (15% ~ 36 minutter) – JavaScript programmering

For måling av trafikkbelastning på en vei skal det lages en webapplikasjon for å registrere biler som passerer et tellepunkt.

Websiden til applikasjonen inneholder to felt. Det første feltet lar bruker registrere en bil og tidspunkt for passering. Skjermbildet under viser hvordan dette feltet kan se ut:



Skjermbilde 1: Felt for å registrere bilpassering

Som vist i Skjermbilde 1 har dagens dato blitt fylt inn av applikasjonen. En bil med registreringsnummer EC54123 skal bli registrert med et passeringstidspunkt 08:22 når knappen *Registrer passering* blir klikket på.

Dersom et av feltene mangler verdi skal ingenting skje om bruker klikker på knappen *Registrer passering*.

Bilpasseringen i Skjermbilde 1 kan bli representert med et objekt på følgende form:

```
const passering = {  
  "regnr": "EC54123",  
  "tidtall": 502,  
  "tidstreng": "08:22"  
};
```

Kode eksempel 1: Bil med registreringsnummer EC54123 passerte tellepunktet klokken 08:22

Parameter *tidtall* er tidspunktet 08:22 omregnet til antall minutter siden midnatt. Denne parameteren gjør det enklere å sammenligne bilpasseringer.

Bilpasseringer må lagres i en liste for senere bruk, for eksempel ved å bruke en forekomst av en JavaScript **Array**.

Det neste feltet lar bruker liste bilpasseringer i et gitt tidsintervall. Skjermbildet under viser hvordan dette feltet kan se ut:

Bilpasseringer

Bilnummer	Tidspunkt	Fra:	Til:
EK12345	08:20	08 : 19	12 : 35
EC54123	08:22		
EB44444	10:15		
EL12345	12:30		

Vis passeringer

Skjermbilde 2: Bilpasseringer fra 08:19 til 12:35

I Skjermbilde 2 har bruker fylt inn et tidsintervall og klikket på knappen *Vis passeringer*. Da skal alle passeringer i det gitte tidsintervallet vises, med eldste passering øverst.

Feltet i Skjermbilde 1 for å registrere en bilpassering er laget med HTML-koden som er vist under:

```
<fieldset id="registrering">
  <legend>Bilpassering <span></span></legend>

  <label for="bilnummer">Registreringsnummer:</label>
  <input type="text" size="10" placeholder="Bilnummer"
    id="bilnummer">

  <label for="tidspunkt">Tidspunkt:</label>
  <input type="time" id="tidspunkt">

  <button type="button">Registrer passering</button>
</fieldset>
```

Kode eksempel 2: HTML-kode for å registrere bilpassering

Feltet i Skjermbilde 2 for å liste bilpasseringer i et gitt tidsintervall er laget med HTML-koden som er vist under:

```
<fieldset id="resultat">
  <legend>Bilpasseringer</legend>
  <div class="input">
    <div>
      <label for="nedregrense">Fra: </label>
      <input type="time" id="nedregrense">
      <label for="ovregrense">Til: </label>
      <input type="time" id="ovregrense">
      <button type="button">Vis passeringer</button>
    </div>
  </div>
  <div>
    <table>
      <thead>
        <tr><th>Bilnummer</th><th>Tidspunkt</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </div>
</fieldset>
```

Kode eksempel 3: HTML-kode for å liste bilpasseringer

Applikasjonen inneholder følgende JavaScript-kode:

```
class Trafikkontroller {
    #nedregrense;
    #ovregrense;
    #tbodyelement;
    #billiste = [];

    constructor(felt1, felt2) {
        // Legg inn kode for felt 1, felt for å registrere bilpassering

        // Legg inn kode for å vise dagens dato i felt1 sitt <legend>

        // Kode for felt2, felt for å liste bilpasseringer
        this.#nedregrense = felt2.getElementsByTagName("input")[0];
        this.#ovregrense = felt2.getElementsByTagName("input")[1];
        this.#tbodyelement = felt2.getElementsByTagName("tbody")[0];
        felt2.querySelector("button").addEventListener(
            "click",
            () => { this.#vispasseringer() }
        );
    }

    #vispasseringer() {
        // Legg inn kode her for å liste bilpasseringer
    }

    // Legg til metode for å registrere en bilpassering

    // Legg til eventuelle hjelpemetoder
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("resultat");

new Trafikkontroller(felt1, felt2);
```

Kode eksempel 4: JavaScript-kode for å registrere og liste bilpasseringer

Oppgave: Fyll inn den manglende koden i JavaScript-klassen **Trafikkontroller**.

Hjelp:

- Kode-eksempelen under oppretter en variabel *datotekst* som vil inneholde teksten «mandag 9. desember, 2024» på dato 9. desember 2024:

```
const now = new Date();
const dagnavn = now.toLocaleDateString(
  navigator.language,
  { weekday: "long" }
);
const dagnr = now.getDate();
const maaned = now.toLocaleDateString(
  navigator.language,
  { month: "long" }
);
const aar = now.getFullYear();
const datotekst = `${dagnavn} ${dagnr}. ${maaned}, ${aar}`;
```

- HTML input-elementer sitt attributt *value* er nåværende verdi til feltet.
- Et felt `<input type="time">` sin *value* er en streng på formatet «hh:mm», for eksempel «08:22».
- Den globale funksjonen *parseInt()* kan lese et heltall fra en **String** og returnerer tallet som et **Number**.
- Kode-eksempelen under fyller en **Array** fra en tekst, der elementene er separert av et kolon:

```
const tidtabell = "08:22".split(":");
```

- Kode-eksempelen under setter inn et nytt element i tabellen *this.#billiste* mellom de eksisterende elementene med indeks 2 og indeks 3:

```
this.#billiste.splice(3, 0, bil);
```

Det nye elementet får indeks 3 og alle påfølgende element øker sin indeks verdi med 1. Altså vil elementet som hadde indeks 3 få indeks 4.

- **Array** sin metode *push()* legger til elementer på slutten av en tabell.
- **Array** sin metode *unshift()* legger til elementer til starten av en tabell.
- Både **Map**, **Set** og **Array** har en metode *forEach*.
- Om **Array** sin metode *findIndex*, fra MDN:

*The `findIndex()` method of **Array** instances returns the index of the first element in an array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.*

- Om **Array** sin metode *filter*, fra MDN:

*The `filter()` method of **Array** instances creates a shallow copy of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.*

Oppgave 6 (10% ~ 24 minutter) – Passord

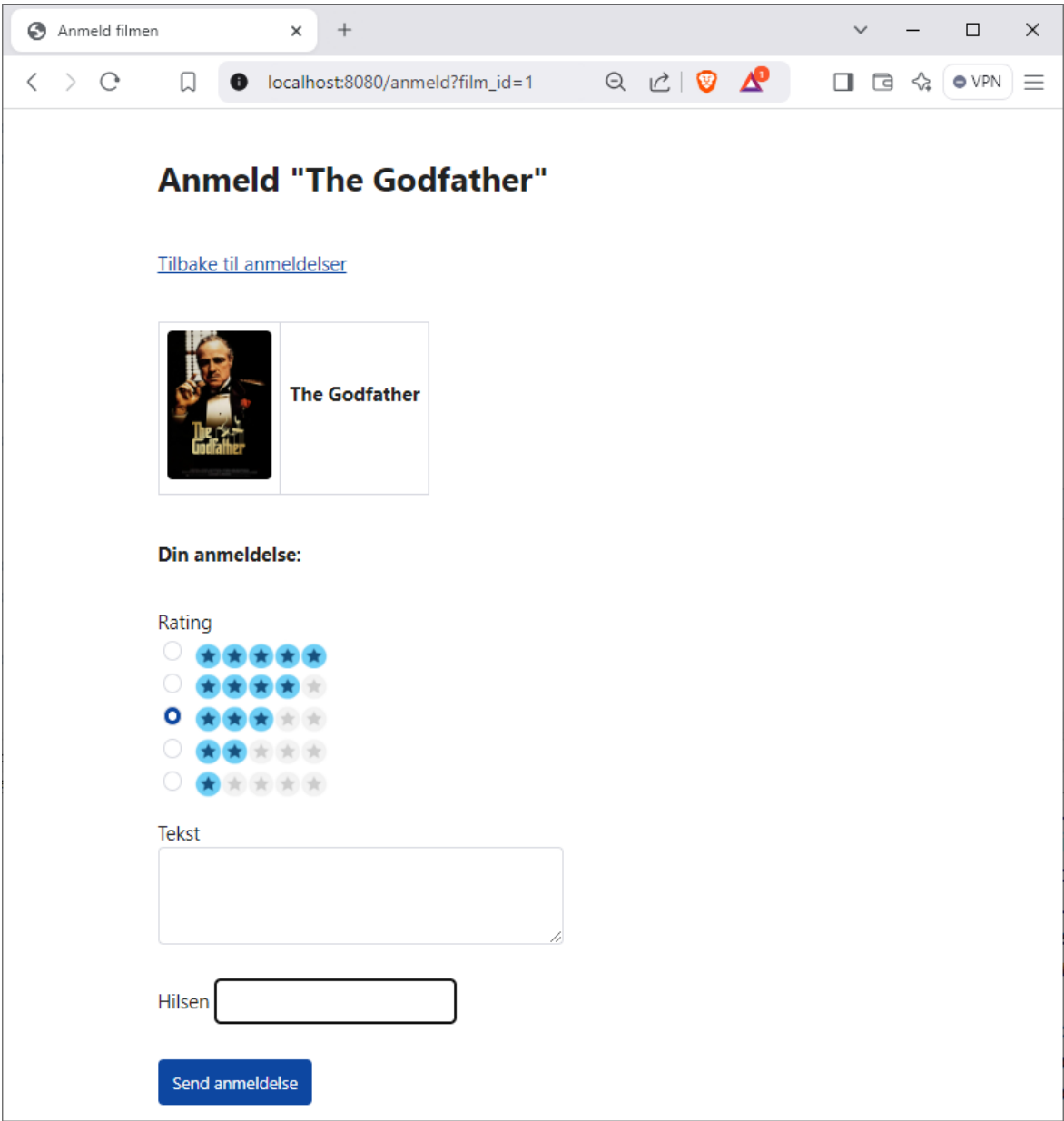
Korrekt passordhåndtering og -lagring er svært viktig både for sikkerhet og personvern.

- a) Hvilke «krav» bør vi sette til brukeres passord, og hvorfor? Fortell hvilke typer angrep ulike krav beskytter mot.
- b) Det anbefales å salte og hashe passord ved lagring. Forklar hva salting er, og hvordan dette bør gjøres. Hvilke typer angrep beskytter salting mot?

Oppgave 7 (30% ~ 72 minutter) – Web backend med Spring MVC

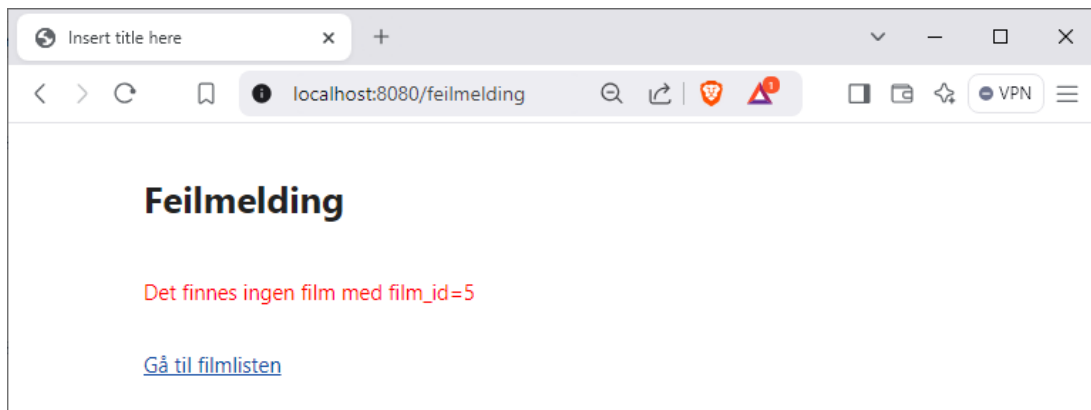
I denne oppgaven skal du jobbe med og lage en liten del av en webapplikasjon for filmanmeldelser. Et par aktuelle URL-er / sider:

- **/filmliste** ... (ikke relevant for oppgavene annet enn at et par sider har en lenke hit)
- **/anmeldelser** ... (ikke relevant ...) Request-parameter er **film_id**.
- **/anmeld** viser skjema for anmeldelse av én film (gitt ved parameteren **film_id**), se bilde 7.1. Ved trykk på knappen "Send anmeldelse" blir anmeldelsen postet til den samme URL-en og lagret i databasen. Deretter omdirigeres man igjen til **anmeldelser** for filmen.
- **/feilmelding** viser (den dynamiske) feilmeldingssiden som kan benyttes i feilsituasjoner, f.eks. ved manglende eller ugyldige parametere, se bilde 7.2.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/anmeld?film_id=1'. The page title is 'Anmeld "The Godfather"'. Below the title is a link 'Tilbake til anmeldelser'. A movie poster for 'The Godfather' is shown next to the title. Underneath is a section titled 'Din anmeldelse:' with a 'Rating' section containing five rows of star icons. The first row has five blue stars, the second has four blue stars and one grey star, the third has three blue stars and two grey stars, the fourth has two blue stars and three grey stars, and the fifth has one blue star and four grey stars. Below the stars is a 'Tekst' label and a text input field. At the bottom is a 'Hilsen' label and a text input field. A blue button labeled 'Send anmeldelse' is at the bottom right.

Bilde 7.1



Bilde 7.2

Litt om dataene i applikasjonen

Vi har lagret data om filmer og anmeldelser i en database, og henting og lagring av data gjøres via Spring-JpaRepositoriene **FilmRepo** og **AnmeldelseRepo**.

Java-entitetsklassen **Film** har disse egenskapene:

- int **id** (primærnøkkel)
- String **tittel** (navnet på filmen)
- String **bildefil** (filnavnet på poster-bildet til filmen)
- List<Anmeldelse> **anmeldelser** (listen av anmeldelser. fetch=EAGER)

I tillegg har Film disse metodene som kan være nyttige:

- void **leggTilAnmeldelse**(Anmeldelse anmeldelse) //Legger til anmeldelse i listen
- double **getSnittratingMedEnDesimal**() //Beregner snittrating fra anmeldelser
- int **getAvrundetSnittrating**() //Beregner snittrating fra anmeldelser, avrundet til heltall

Java-entitetsklassen **Anmeldelse** har disse egenskapene:

- String **id** (autogenerert primærnøkkel)
- int **rating** (rating i antall stjerner, fra 1 til 5)
- String **tekst** (selve anmeldelsesteksten)
- String **anmelder** (navnet på anmelder)
- Film **film** (referanse til filmen som anmeldelsen tilhører)

Du kan anta at disse klassene har de nødvendige konstruktører, get- og set-metoder.

I denne oppgaven trenger du ingen @Service mellom Controller og Repositories. Controlleren kan bruke repositoriene direkte, og klarer seg med et par repository-metoder:

- Optional<Film> **filmRepo.findById**(int film_id)
- Anmeldelse **anmeldelseRepo.save**(Anmeldelse anmeldelse)

Anmeldelser for en gitt film kan du hente ut som en egenskap fra film-objektet.

Validering av brukerinput

En `InputValidator` kan brukes til å validere brukerinput for anmeldelsen. Metoden `erGyldigInput()` vil validere både rating, tekst og anmelder.

Validatoren har i tillegg en metode `erStringEtPositivtHeltall()` som f.eks. kan brukes til å sjekke at requestparameteren `film_id` er `!= null` og er et positivt heltall (hvis du velger å la denne parameteren være en `String`).

```
@Service
public class InputValidator {
    public boolean erGyldigInput(Anmeldelse a) {...}
    public boolean erStringEtPositivtHeltall(String s) {...}
}
```

Oppgaver

Oppgavene skal løses med Spring MVC og JSP m/EL og JSTL, og ellers i henhold til prinsippene gjennomgått i kurset, som f.eks. inputvalidering av all brukerinput, Post-Redirect-Get, osv...

Du skal i oppgaven fokusere på å anmelde en film. Controlleren er definert slik:

```
@Controller
public class AnmeldController {

    @Autowired FilmRepo filmRepo;
    @Autowired AnmeldelseRepo anmeldelseRepo;

    @Autowired InputValidator inputValidator;

    ...
}
```

- a) Skriv metoden i **AnmeldController** for å hente frem skjema for å anmelde, som vist i bilde7.1.

Tips:

- Husk å sjekke at **film_id** er gyldig. Hva om `film_id` mangler? Hva om `film_id` ikke er et heltall? Hva om det ikke finnes en film med denne id-en i databasen. Alle mulige feil må håndteres.
- Plusspoeng om det tas hensyn til at søkeresultat er på formen **Optional<Film>**.
- View-komponenten tilhørende denne siden er **anmeld.jsp**.

b) Skriv tilhørende view, altså JSP-siden **anmeld.jsp**.

Tips:

- Du kan anta at bilder ligger i mappen **img**.
- Du kan anta at bildene for rating heter **rating1.png**, **rating2.png**, osv...
- Plusspoeng om du klarer å sette opp rating-bildene med en `<forEach>`-løkke.

c) Skriv metoden i `AnmeldController` for å ta imot innsendt anmeldelse.

Tips:

- Husk å sjekke at `film_id` er gyldig på samme måte som i a)
- Husk å validere all brukerinput. Bruk hjelpeklassen **`InputValidator`**.
- Se beskrivelsen helt i starten om hvor man skal havne etterpå.

d) Vi skal nå legge til litt ekstra funksjonalitet. Tenk at du ofte er inne og anmelder filmer. Hvordan kan vi gjøre slik at du slipper å skrive inn navnet ditt hver gang? Jo, vi kan bruke **cookies**.

Ved anmeldelse av en film skal anmelders navn lagres som en cookie i nettleseren. Neste gang du vil anmelde en film, skal dette navnet settes automatisk inn i input-feltet for anmelders navn på nettsiden.

Du kan velge selv om du blander løsningen inn i svarene for a), b) og c), eller om du lager et separat svar for d) som kun forteller hvor og hvordan ting må endres for å få denne funksjonaliteten.

Til å hjelpe deg har du hjelpeklassen `CookieService`.

`CookieService` kan brukes både ved mottak av cookies og ved sending av cookies. Metodene `getCookieFromRequest()` og `addCookieToResponse()` vil hhv. URL-dekode og URL-kode cookie-verdien som sendes over HTTP. Velger du å ikke bruke disse metodene, må URL-dekoding og URL-koding gjøres separat.

```
@Service
public class CookieService {

    public String getCookieFromRequest(
        HttpServletRequest request, String cookieName) {...}

    public void addCookieToResponse(HttpServletResponse response,
        String cookieName, String cookieValue) {...}

    public String urlEncode(String cookieValue) {...}
    public String urlDecode(String cookieValue) {...}
}
```

Slutt på oppgavesettet

EKSAMENSOPPGÅVE

Nynorsk

Emnekode: DAT108

Emnenamn: Programmering og webapplikasjoner

Klasse: 2. klasse DATA / INF

Dato: 9. desember 2024

Eksamensform: Skriftleg skuleeksamen (Wiseflow | FLOWlock)

Eksamenstid: 4 timer (0900-1300)

Tal på eksamensoppgåver: 7

Tal på sider (medrekna denne): 15

Tal på vedlegg: Ingen

Tillate hjelpemiddel: Ingen

Lærarar: Lars-Petter Helland (928 28 046), Bergen
 Bjarte Kileng (909 97 348), Bergen
 Erlend Raa Vågset (910 06 061), Førde
 Tord Waage Frøland (915 45 537), Haugesund

LUKKE TIL!

Oppgave 1 (10% ~ 24 minutt) – Straumar

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutt) – Funksjonelle kontraktar

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow

Oppgave 3 (10% ~ 24 minutt) – JavaScript teori

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow



Felles info for oppgave 4, 5, 6 og 7

Du skal svare på desse i Wiseflow. Ved å bruke kodevedlegg vil de få utheva syntaks og korrekte innrykk. Vi tilrår eitt vedlegg per oppgåve. For å sikre automatisk lagring, følg denne oppskrifta.

- 1 - Start med å gi vedlegget eit namn. Til dømes «Oppgave 4.java».
- 2 - Klikk lagre. Då aktiverer du automatisk lagring.
- 3 - Start å skrive kode

Pass på at du har svart på alle oppgåver før du prøver å få oppgåver perfekte. Viss du meiner noko er uklart i oppgåva, presiser føresetnadene dine i svaret.

Oppgåvene byrjar på neste side.

Oppgåve 4 (15% ~ 36 minutt) – Trådar og trådsikkerhet

Nedanfor er det gitt eit lite Java-program som simulerer ein vintermarknad der to Gløggkokarar og to Pepperkakebakarar jobbar parallelt for å laga høvesvis gløgg og peparkaker.

Kvar arbeidar bruker 2 sekund for å laga éi eining gløgg eller peparkake, og legg deretter produktet på eit felles serveringsbrett: gløggserveringsbrett eller peparkakeserveringsbrett.

Desse serveringsbretta blir delte mellom arbeidarane av same type, slik at Gløggkokarar og Pepperkakebakarar legg varene sine på høvesvis felles gløggserveringsbrett og peparkakeserveringsbrett.

Tre kundar besøker marknaden. Kvar kunde vel tilfeldig mellom gløgg og peparkaker, og tek frå brettet når vara er tilgjengeleg. Viss eit brett er tomt, ventar kunden før dei prøver igjen. Alle arbeidarane og kundane køyrer parallelt – kundane ventar berre på at ei vare skal bli tilgjengeleg, ikkje på kvarandre.

Koden inneheld nokon feil relaterte til trådbehandling og trådsikring. Oppgåvene vil rettleia deg gjennom å identifisera og rette desse feila.

Oppgåver

- a) **Køyring av Arbeidar- og Kundetrådar.** I Vintermarked sin main-metode er arbeidar- og kundetrådane definert.
 - i. Identifiser problem med måten trådane er starta i main-metoden.
 - ii. Endre main-metoden slik at alle trådar køyrer parallelt som forventa. (Svaret ditt treng berre å ha med den delen av koden som er endra.)
- b) **Trådsikring for Serveringsbretta.** I koden er gløggServeringsbrett og pepperkakeServeringsbrett implementert som Queue-objekt.
 - i. Forklar kort kva som kan skje viss fleire trådar får tilgang til serveringsbretta samtidig utan trådsikring.
 - ii. Endre koden slik at bruken av serveringsbretta blir trådsikker. (Svaret ditt treng berre å ha med den delen av koden som er endra.)
- c) **Synkronisering av Teljarar.** Koden inneheld følgjande teljarar:
 - Globale teljarar for det samla talet produserte og konsumerte varer, som alle arbeidarane og kundane oppdaterer.
 - gloeggServert for det samla talet gløgg servert.
 - pepperkakerServert for det samla talet peparkaker servert.
 - Objektspesifikke teljarar i kvar Arbeider- og Kunde-instans som sporar kor mange varer dei individuelt har produsert eller konsumert.
 - produkterLaget i Arbeider for talet på varer produsert av kvar enkelt arbeidar.
 - varerKonsumert i Kunde for talet på varer konsumert av kvar enkelt kunde.

Vurder kva for nokre av teljarane som treng trådsikring og kva for nokre av dei som ikkje gjer det, og forklar kort kvifor du meiner det er nødvendig eller unødvendig.

- d) **Trådsikring av Teljarar.** For dei teljarane du identifiserte i førre oppgåve som krev trådsikring:
- Endre koden slik at desse teljarane handterer trådsikker oppdatering. Vel ein metode du meiner er formålstenleg (til dømes synchronized-blokker, Lock-objekt eller AtomicInteger).
 - Forklar kort kvifor du valde denne metoden.

e) **Forklare Dødlås eller Svolt.**

Vel eitt av følgjande konsept og forklar kort kva det inneber:

- Dødlås (Deadlock): Beskriv kva dødlås er og gi eit døme.
- Svolt/utsvelting (Starvation): Beskriv kva svolt/utsvelting er og gi eit døme.

Start svaret ditt ved å oppgi kva for eit av de to konseptene du forklarer.

Kode:

```
public enum Produkt { GLOEGG, PEPPERKAKE; }
```

```
public class Vintermarked {

    // To felles serveringsbrett for hhv. gløgg og pepperkaker
    public static final Queue<String> gloeggServeringsbrett = new LinkedList<>();
    public static final Queue<String> pepperkakeServeringsbrett = new LinkedList<>();

    // Globale tellere for totalt antall servert
    public static int gloeggServert = 0;
    public static int pepperkakerServert = 0;

    public static void main(String[] args) {

        // Oppretter tråder for Gløggkokere og Pepperkakebakere
        Arbeider gloeggkoker1 = new Arbeider(
            "Gløggkoker-1", Produkt.GLOEGG, gloeggServeringsbrett);
        Arbeider gloeggkoker2 = new Arbeider(
            "Gløggkoker-2", Produkt.GLOEGG, gloeggServeringsbrett);
        Arbeider pepperkakebaker1 = new Arbeider(
            "Pepperkakebaker-1", Produkt.PEPPERKAKE, pepperkakeServeringsbrett);
        Arbeider pepperkakebaker2 = new Arbeider(
            "Pepperkakebaker-2", Produkt.PEPPERKAKE, pepperkakeServeringsbrett);

        // Starter arbeidernes prosesser
        gloeggkoker1.run();
        gloeggkoker2.run();
        pepperkakebaker1.run();
        pepperkakebaker2.run();

        // Oppretter og starter tråder for 3 kunder
        for (int i = 0; i < 3; i++) {
            new Thread(new Kunde(i)).start();
        }
    }
}
```

```

public class Arbeider implements Runnable {

    private final String navn;
    private final Produkt produkt;
    private final Queue<String> tilknyttetBrett;
    private int produkterLaget = 0;

    /* Konstruktør fjernet - Du kan anta at den finnes */

    @Override
    public void run() {
        try {
            while (true) {
                Thread.sleep(2000); // Produksjonstid på 2 sek

                tilknyttetBrett.add(produkt + " laget av " + navn);
                if (produkt == Produkt.GLOEGG) {
                    Vintermarked.gloeggServert++
                } else {
                    Vintermarked.pepperkakerServert++;
                }
                produkterLaget++;
                System.out.println(navn + " la til en "+ produkt +" på brettet. "
                    + "Totalt produsert av denne arbeideren: "
                    + produkterLaget);
            }
        } catch (InterruptedException e) { ... }
    }
}

```

```

public class Kunde implements Runnable {

    private final int id; // Unik ID for kunden
    private int varerKonsumert = 0;

    /* Konstruktør fjernet - Du kan anta at den finnes */

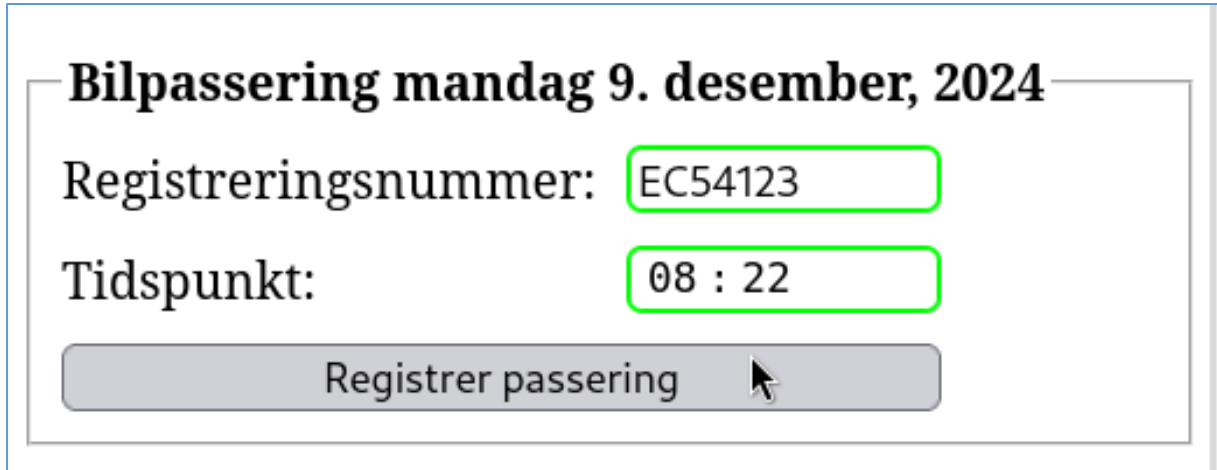
    @Override
    public void run() {
        try {
            while (true) {
                // Forsøker å ta gløgg fra brettet
                String vare = Vintermarked.gloeggServeringsbrett.poll();
                // Hvis tomt, forsøker å ta pepperkake
                if (vare == null) {
                    vare = Vintermarked.pepperkakeServeringsbrett.poll();
                }
                if (vare != null) {
                    varerKonsumert++;
                    System.out.println("Kunde " + id + " konsumerte: " + vare
                        + ". Totalt konsumert av denne kunden: "
                        + varerKonsumert);
                    Thread.sleep(1500); // Simulerer konsumtid på 1,5 sek
                } else {
                    System.out.println("Kunde " + id + " venter på vare...");
                    Thread.sleep(500); // Venter 0,5 sek hvis ingen varer er tilgjengelige
                }
            }
        } catch (InterruptedException e) { ... }
    }
}

```

Oppgave 5 (15% ~ 36 minutt) – JavaScript programmering

For måling av trafikkbelastning på ein veg skal det verte laga ein webapplikasjon for å registrera bilar som passerer eit teljepunkt.

Websida til applikasjonen inneheld to felt. Det fyrste feltet lar brukar registrera ein bil og tidspunkt for passering. Skjermbilete under viser korleis dette feltet kan sjå ut:



Skjermbilete 1: Felt for å registrera bilpassering

Som vist i Skjermbilete 1 **Feil! Fant ikke referanse kilden.** har dagen sin dato verte fylt inn av applikasjonen. Ein bil med registreringsnummer EC54123 skal verte registrert med eit passeringstidspunkt 08:22 når knappen *Registrer passering* verte klikka på.

Dersom eit av felta manglar verte skal ingenting skje om brukar klikkar på knappen *Registrer passering*.

Bilpasseringa i Skjermbilete 1 kan verte representert med eit objekt på fylgjande form:

```
const passering = {  
  "regnr": "EC54123",  
  "tidtall": 502,  
  "tidstreng": "08:22"  
};
```

Kode døme 1: Bil med registreringsnummer EC54123 passerte tellepunktet klokka 08:22

Parameter *tidtall* er tidspunktet 08:22 omrekna til tal på minutt sia midnatt. Denne parameteren gjer det enklare å samanlikne bilpasseringar.

Bilpasseringar må verte lagra i ein liste for seinare bruk, til dømes ved å nytte ein førekomst av ein JavaScript **Array**.

Det neste feltet lar brukar liste bilpasseringar i eit gjeve tidsintervall. Skjermbiletet under viser korleis dette feltet kan sjå ut:

Bilpasseringer

Bilnummer	Tidspunkt	Fra:	Til:
EK12345	08:20	08 : 19	12 : 35
EC54123	08:22		
EB44444	10:15		
EL12345	12:30		

Vis passeringer

Skjermbilete 2: Bilpasseringer fra 08:19 til 12:35

I Skjermbilete 2 har brukar fylt inn eit tidsintervall og klikka på knappen *Vis passeringer*. Då skal alle passeringar i det gjeve tidsintervallet verte vist, med eldste passering fyrst.

Feltet i Skjermbilete 1 for å registrera ein bilpassering er laga med HTML-koden som er vist under:

```
<fieldset id="registrering">
  <legend>Bilpassering <span></span></legend>

  <label for="bilnummer">Registreringsnummer:</label>
  <input type="text" size="10" placeholder="Bilnummer"
    id="bilnummer">

  <label for="tidspunkt">Tidspunkt:</label>
  <input type="time" id="tidspunkt">

  <button type="button">Registrer passering</button>
</fieldset>
```

Kode dørne 2: HTML-kode for å registrera bilpassering

Feltet i Skjermbilete 2 for å liste bilpasseringar i eit gjeve tidsintervall er laga med HTML-koden som er vist under:

```
<fieldset id="resultat">
  <legend>Bilpasseringer</legend>
  <div class="input">
    <div>
      <label for="nedregrense">Fra: </label>
      <input type="time" id="nedregrense">
      <label for="ovregrense">Til: </label>
      <input type="time" id="ovregrense">
      <button type="button">Vis passeringer</button>
    </div>
  </div>
  <div>
    <table>
      <thead>
        <tr><th>Bilnummer</th><th>Tidspunkt</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </div>
</fieldset>
```

Kode dørne 3: HTML-kode for å liste bilpasseringar

Applikasjonen inneholdt følgende JavaScript-kode:

```
class Trafikkontroller {
    #nedregrense;
    #ovregrense;
    #tbodyelement;
    #billiste = [];

    constructor(felt1, felt2) {
        // Legg inn kode for felt 1, felt for å registrere bilpassering

        // Legg inn kode for å vise dagens dato i felt1 sitt <legend>

        // Kode for felt2, felt for å liste bilpasseringer
        this.#nedregrense = felt2.getElementsByTagName("input")[0];
        this.#ovregrense = felt2.getElementsByTagName("input")[1];
        this.#tbodyelement = felt2.getElementsByTagName("tbody")[0];
        felt2.querySelector("button").addEventListener(
            "click",
            () => { this.#vispasseringer() }
        );
    }

    #vispasseringer() {
        // Legg inn kode her for å liste bilpasseringer
    }

    // Legg til metode for å registrere en bilpassering

    // Legg til eventuelle hjelpemetoder
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("resultat");

new Trafikkontroller(felt1, felt2);
```

Kode dørne 4: JavaScript-kode for å registrere og liste bilpasseringar

Oppgave: Fyll inn den manglende koden i JavaScript-klassa **Trafikkontroller**.

Hjelp:

- Døme nedanfor med kode opprettar ein variabel *datotekst* som vil innehalde teksten «mandag 9. desember, 2024» på dato 9. desember 2024:

```
const now = new Date();
const dagnavn = now.toLocaleDateString(
  navigator.language,
  { weekday: "long" }
);
const dagnr = now.getDate();
const maaned = now.toLocaleDateString(
  navigator.language,
  { month: "long" }
);
const aar = now.getFullYear();
const datotekst = `${dagnavn} ${dagnr}. ${maaned}, ${aar}`;
```

- HTML input-element sitt attributt *value* er nåverande verde til feltet.
- Eit felt `<input type="time">` sin *value* er ein streng på formatet «hh:mm», til dømes «08:22».
- Den globale funksjonen *parseInt()* kan lese eit heiltal i ein **String** og returnere talet som eit **Number**.
- Kode-eksempelen under fyller en **Array** fra en tekst, der elementene er separert av et kolon:

```
const tidtabell = "08:22".split(":");
```

- Døme nedanfor med kode set inn eit nytt element i tabellen *this.#billiste* mellom de eksisterande elementa med indeks 2 og indeks 3:

```
this.#billiste.splice(3, 0, bil);
```

Det nye elementet får indeks 3 og alle påfølgjande element aukar sitt indeks verde med 1. Altså vil elementet som hadde indeks 3 få indeks 4.

- **Array** sin metode *push()* legg til element på slutten av ein tabell.
- **Array** sin metode *unshift()* legg til element til starten av ein tabell.
- Både **Map**, **Set** og **Array** har ein metode *forEach*.
- Om **Array** sin metode *findIndex*, frå MDN:

*The `findIndex()` method of **Array** instances returns the index of the first element in an array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.*

- Om **Array** sin metode *filter*, frå MDN:

*The `filter()` method of **Array** instances creates a shallow copy of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.*

Oppgave 6 (10% ~ 24 minutt) – Passord

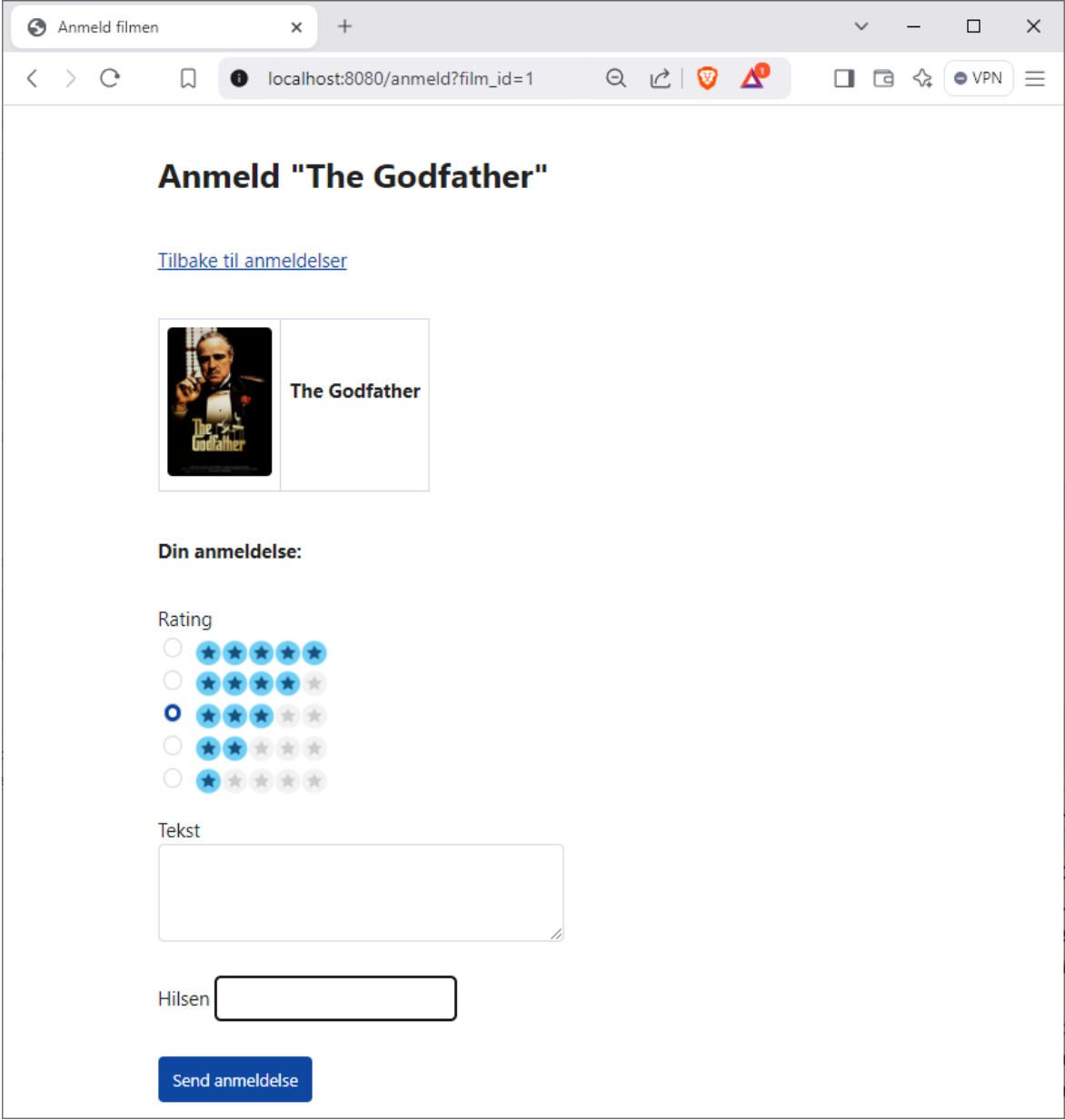
Korrekt passordhandtering og -lagring er svært viktig både for tryggleik og personvern.

- a) Kva «krav» bør me setja til brukarars passord, og kvifor? Fortel kva typar angrep ulike krav vernar mot.
- b) Det blir tilrådd å salta og hashe passord ved lagring. Forklar kva salting er, og korleis dette bør gjerast. Kva typar angrep vernar salting mot?

Oppgave 7 (30% ~ 72 minutt) – Web backend med Spring MVC

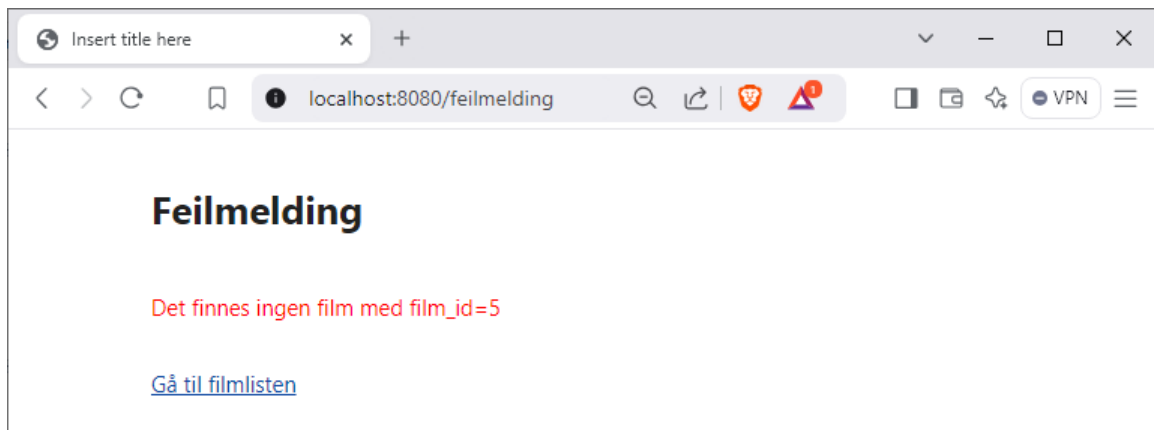
I denne oppgåva skal du jobba med og laga ein liten del av ein webapplikasjon for filmmeldingar (*bm. filmanmeldelser*). Eit par aktuelle URL-er / sider:

- **/filmliste** ... (ikkje relevant for oppgåvene anna enn at eit par sider har ei lenkje hit)
- **/anmeldelser** ... (ikkje relevant ...) Request-parameter er `film_id`.
- **/anmeld** viser skjema for melding av éin film (gitt ved parameteren **film_id**), sjå bilde 7.1. Ved trykk på knappen "Send anmeldelse" blir meldinga posta til den same URL-en og lagra i databasen. Deretter blir ein omdirigert igjen til **anmeldelser** for filmen.
- **/feilmelding** viser (den dynamiske) feilmeldingssida som kan nyttast i feilsituasjonar, t.d. ved manglande eller ugyldige parametarar, sjå bilde 7.2.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/anmeld?film_id=1'. The page title is 'Anmeld "The Godfather"'. Below the title is a link 'Tilbake til anmeldelser'. A movie poster for 'The Godfather' is shown next to the title. Underneath is a section titled 'Din anmeldelse:' with a 'Rating' section containing five rows of star icons. The third row has the third star selected. Below the rating is a 'Tekst' input field. At the bottom is a 'Hilsen' input field and a blue button labeled 'Send anmeldelse'.

Bilde 7.1



Bilde 7.2

Litt om data i applikasjonen

Vi har lagra data om filmar og meldingar i ein database, og henting og lagring av data blir gjort via Spring-JpaRepositoriene **FilmRepo** og **AnmeldelseRepo**.

Java-entitetsklassen **Film** har desse eigenskapane:

- int **id** (primærnøkkel)
- String **tittel** (namnet på filmen)
- String **bildefil** (filnamnet på poster-biletet til filmen)
- List<Anmeldelse> **anmeldelser** (lista av meldingar. fetch=EAGER)

I tillegg har Film desse metodane som kan vera nyttige:

- void **leggTilAnmeldelse**(Anmeldelse anmeldelse) //Legg til melding i lista
- double **getSnittratingMedEnDesimal**() //Beregner snittrating frå meldingar
- int **getAvrundetSnittrating**() //Beregner snittrating frå meldingar, avrunda til heiltal

Java-entitetsklassen **Anmeldelse** har desse eigenskapane:

- String **id** (autogenerert primærnøkkel)
- int **rating** (rating i talet på stjerner, frå 1 til 5)
- String **tekst** (sjølve meldingsteksten)
- String **anmelder** (namnet på melder)
- Film **film** (referanse til filmen som meldinga tilhøyrer)

Du kan rekne med at desse klassane har dei nødvendige konstruktørar, get- og set-metodar.

I denne oppgåva treng du ingen @Service mellom Controller og Repositories. Controlleren kan bruka repositoria direkte, og klarer seg med eit par repository-metodar:

- Optional<Film> **filmRepo.findById**(int film_id)
- Anmeldelse **anmeldelseRepo.save**(Anmeldelse anmeldelse)

Meldingar for ein gitt film kan du henta ut som en eigenskap frå film-objektet.

Validering av brukarinput

En `InputValidator` kan brukast til å validera brukarinput for meldinga. Metoden `erGyldigInput()` vil validera både rating, tekst og namn på melder.

Validatoren har i tillegg ein metode `erStringEtPositivtHeiltall()` som t.d. kan brukast til å sjekka at requestparameteren `film_id` er `!= null` og er eit positivt heiltal (viss du vel å la denne parameteren vera ein `String`).

```
@Service
public class InputValidator {
    public boolean erGyldigInput(Anmeldelse a) {...}
    public boolean erStringEtPositivtHeiltall(String s) {...}
}
```

Oppgåver

Oppgåvene skal løysast med Spring MVC og JSP m/EL og JSTL, og elles i samsvar med prinsippa gått gjennom i kurset, som t.d. inputvalidering av all brukarinput, Post-Redirect-Get, osv...

Du skal i oppgåva fokusere på å melde ein film. Controllaren er definert slik:

```
@Controller
public class AnmeldController {

    @Autowired FilmRepo filmRepo;
    @Autowired AnmeldelseRepo anmRepo;

    @Autowired InputValidator inputValidator;

    ...
}
```

- a) Skriv metoden i `AnmeldController` for å henta frem skjema for å melda, som vist i bilde7.1.

Tips:

- Hugs å sjekka at **film_id** er gyldig. Kva om `film_id` manglar? Kva om `film_id` ikkje er eit heiltal? Kva om det ikkje finst ein film med denne id-en i databasen. Alle moglege feil må handterast.
- Plusspoeng om det blir teke omsyn til at søkeresultat er på formen **Optional<Film>**.
- View-komponenten tilhøyrande denne sida er **anmeld.jsp**.

b) Skriv tilhørende view, altså JSP-sida **anmeld.jsp**.

Tips:

- Du kan rekna med at bilete ligg i mappa **img**.
- Du kan rekna med at bileta for rating heiter **rating1.png**, **rating2.png**, osv...
- Plusspoeng om du klarer å setja opp rating-bileta med ein <forEach>-løkke.

c) Skriv metoden i AnmeldController for å ta imot innsend melding.

Tips:

- Hugs å sjekka at film_id er gyldig på same måte som i a)
- Hugs å validera all brukarinput. Bruk hjelpeklassen **InputValidator**.
- Sjå skildringa heilt i starten om kvar ein skal hamna etterpå.

d) Vi skal no leggja til litt ekstra funksjonalitet. Tenk at du ofte er inne og melder filmar. Korleis kan vi gjera slik at du slepp å skriva inn namnet ditt kvar gong? Jo, vi kan bruka **cookies**.

Ved melding av ein film skal meldars namn lagrast som ein cookie i nettlesaren. Neste gong du vil melda ein film, skal dette namnet setjast automatisk inn i input-feltet for meldars namn på nettsida.

Du kan velja sjølv om du blandar løysinga inn i svara for a), b) og c), eller om du lagar eit separat svar for d) som berre fortel kvar og korleis ting må endrast for å få denne funksjonaliteten.

Til å hjelpa deg har du hjelpeklassen **CookieService**.

CookieService kan brukast både ved mottak av cookies og ved sending av cookies. Metodane `getCookieFromRequest()` og `addCookieToResponse()` vil hhv. URL-dekode og URL-kode cookie-verdien som blir send over HTTP. Vel du å ikkje bruke desse metodane, må URL-dekoding og URL-koding bli gjort separat.

```
@Service
public class CookieService {

    public String getCookieFromRequest(
        HttpServletRequest request, String cookieName) {...}

    public void addCookieToResponse(HttpServletResponse response,
        String cookieName, String cookieValue) {...}

    public String urlEncode(String cookieValue) {...}
    public String urlDecode(String cookieValue) {...}
}
```

Slutt på oppgåvesettet